RANDAO-manipuláció elleni védekezés: időzáras titkosítás

1 RANDAO-manipuláció elleni védekezés: időzáras titkosítás

A forking támadás mélyebb oka, hogy a támadó *előre látja*, mely RANDAO-kimenet hány jövőbeli slotot oszt neki, és így a kihagyásról vagy forkolásról tájékozott döntést hozhat. Ha ezt a jövőbeli információt el lehetne rejteni a döntés pillanatáig, a manipuláció elveszítené az alapját. Erre kínál megoldást az időzáras titkosítás (timelock encryption, azaz olyan titkosítás, amelynek titkos szövege csak egy előre megadott idő elteltével fejthető vissza): a validátorok a hozzájárulásukat időzárral titkosítva adhatnák le, így a kimenet csak akkor válik ismertté, amikor már nem lehet rajta változtatni.

A *tlock* séma [**?**] egy gyakorlatban is működő időzáras titkosítás, amely egy küszöb- (threshold-) hálózatra épül. Az alapötlet, hogy a jövőbeli visszafejtő kulcsot egy küszöb-aláírás szolgáltatja: a hálózat minden időszelethez (round, azaz időintervallum) közzétesz egy BLS-aláírást, és ez az aláírás pontosan az adott időhöz tartozó privát kulcs. A konstrukció Boneh és Franklin azonosság-alapú titkosítását (identity-based encryption, azaz nyilvános kulcs helyett egy nyilvános azonosságra – itt az időszelet sorszámára – titkosító séma) [**?**] használja, ahol az azonosság szerepét az időszelet sorszáma tölti be.

A protokoll primitívei. A séma egy III-as típusú bilineáris csoporton (e, G_1, G_2, G_T, p) dolgozik, ahol $e : G_1 \times G_2 \to G_T$ a (nem-degenerált, hatékonyan számítható) párosítás, és négy kriptográfiai hash-függvényt (ütközés- és ősképálló leképezést) használ:

$$H_1 : \{0,1\}^* \to G_2, \qquad H_2 : G_T \to \{0,1\}^\ell,$$
$$H_3 : \{0,1\}^\ell \times \{0,1\}^\ell \to \mathbb{Z}_p^*, \qquad H_4 : \{0,1\}^\ell \to \{0,1\}^\ell.$$

A hálózatot négy eljárás írja le; az első kettő a küszöb-hálózat elosztott (distributed, azaz több fél között megosztott) protokollja, az utóbbi kettő bárki által önállóan futtatható:

- **tSetUp** (elosztott kulcsgenerálás, DKG): a n résztvevő a Gennaro és társai által javasolt elosztott kulcsgenerálási protokollt [**?**] futtatja. Ennek végén minden i fél csak a saját s_i titkos részét (share, azaz megosztás-darab) ismeri a s közös titkos kulcsból, a nyilvános kimenet pedig a hálózati nyilvános kulcs $P = sG_1 \in G_1$. A s kulcs sosem áll elő egyben; bármely legalább t elemű I részhalmaz Lagrange-interpolációval rekonstruálhatja a hozzá tartozó aláírást.

- **tRoundKey** (időszelet-kulcs protokoll): ez maga a küszöb-BLS [?] aláírás a ρ időszelet sorszámára. Minden i fél kiszámítja a részaláírását $\pi_\rho^{(i)} = s_i H_1(\rho)$, majd t darab (ellenőrzött) részaláírásból a teljes időszelet-kulcs

$$\pi_\rho = \sum_{i \in I} \pi_\rho^{(i)} L_i(0) = s H_1(\rho) \in G_2 \tag{1}$$

  áll elő, ahol L_i az I halmazon értelmezett i-edik Lagrange-polinom. Ezt a hálózat időszeletenként közzéteszi.

- **Encrypt** és **Decrypt**: a titkosítás és visszafejtés a Boneh–Franklin séma választott-titkosszöveg-biztos (CCA-biztos) változata, lásd a **??**. és **??**. algoritmust.

A kulcsgondolat, hogy a titkosító a $\mathrm{PK}_\rho = e(P, H_1(\rho))$ „időszeletre szóló nyilvános kulcsot" a nyilvános P-ből és az időszelet sorszámából önállóan ki tudja számítani – a visszafejtéshez viszont a π_ρ időszelet-kulcsra van szükség, amelyet a hálózat csak az adott idő elértekor tesz közzé. A $e(U, \pi_\rho) = e(G_1, H_1(\rho))^{rs} = e(P, H_1(\rho))^r$ egyenlőség (a párosítás bilinearitása miatt) garantálja, hogy a két oldalon ugyanaz a megosztott titok áll elő.

Algoritmus 1 tlock – titkosítás egy ρ időszeletre

procedure ENC(pp, ρ, M) ▷ $M \in \{0,1\}^\ell$ az üzenet
  $\mathrm{PK}_\rho \leftarrow e(P, H_1(\rho))$ ▷ időszeletre szóló nyilvános kulcs
  $\sigma \leftarrow_\$ \{0,1\}^\ell$ ▷ véletlen egyszer használatos érték (nonce)
  $r \leftarrow H_3(\sigma, M)$
  $U \leftarrow rG_1$ ▷ efemer (egyszer használatos) nyilvános kulcs
  $V \leftarrow \sigma \oplus H_2(\mathrm{PK}_\rho^r)$ ▷ a σ elrejtése (commitment)
  $W \leftarrow M \oplus H_4(\sigma)$ ▷ egyszeri kulcsú (one-time-pad) rejtjelezés
  return $\mathrm{ct}_\rho = (U, V, W)$
end procedure

Algoritmus 2 tlock – visszafejtés a π_ρ időszelet-kulccsal

procedure DEC($pp, \rho, \pi_\rho, \mathrm{ct}_\rho$)
  $(U, V, W) \leftarrow \mathrm{ct}_\rho$
  $\sigma' \leftarrow V \oplus H_2(e(U, \pi_\rho))$
  $M' \leftarrow W \oplus H_4(\sigma')$
  $r \leftarrow H_3(\sigma', M')$
  if $U = rG_1$ **then** ▷ a commitment ellenőrzése
    return M'
  else
    return $\perp$
  end if
end procedure

A séma egyik gyakorlati előnye, hogy a küszöb-hálózatnak nem kell tudnia a titkosításról: csak a megszokott módon közzéteszi az időszeletenkénti aláírásait, a titkosítást és visszafejtést pedig bárki elvégezheti. A RANDAO-manipuláció elleni védelem így abból fakad, hogy ha a validátorok hozzájárulásai a leadás pillanatában még

időzárral titkosítottak, a támadó a döntésekor nem ismeri a jövőbeli javaslói listát, ezért a kihagyás vagy a forkolás kimenete sem jósolható meg – a manipulációnak megszűnik a várható haszna. Ugyanez az elv alkalmazható a titkosított tranzakció-várólistákra (encrypted mempool, azaz a feldolgozásra váró, de titkosított tranzakciók halmaza) is a MEV és a front-running (versenyelőny-szerzés mások tranzakciói elé sorolással) ellen.

A gyakorlati megvalósítás egy létező küszöb-hálózatra, a League of Entropy nevű rendszerre épül: ez egy 2020 óta üzemelő, egymásban csak korlátozottan megbízó szervezetek (vállalatok, egyetemek, alapítványok) által működtetett, több kontinensre elosztott csomópontokból álló nyilvános véletlenség-forrás (randomness beacon), amely a drand szoftverrel néhány másodpercenként küszöb-BLS-aláírt véletlen értéket tesz közzé, és így a tlock sémához szükséges időszelet-kulcsokat szolgáltatja.

Hivatkozások

[1] N. Gailly, K. Melissaris, Y. Romailler, *tlock: Practical Timelock Encryption from Threshold BLS*, IACR Cryptology ePrint Archive, Report 2023/189, 2023.

[2] D. Boneh, M. Franklin, *Identity-Based Encryption from the Weil Pairing*, Advances in Cryptology – CRYPTO 2001, Lecture Notes in Computer Science, vol. 2139, Springer, pp. 213–229, 2001.

[3] R. Gennaro, S. Jarecki, H. Krawczyk, T. Rabin, *Secure Distributed Key Generation for Discrete-Log Based Cryptosystems*, Advances in Cryptology – EUROCRYPT '99, Lecture Notes in Computer Science, vol. 1592, Springer, pp. 295–310, 1999.

[4] A. Boldyreva, *Threshold Signatures, Multisignatures and Blind Signatures Based on the Gap-Diffie-Hellman-Group Signature Scheme*, Public Key Cryptography – PKC 2003, Lecture Notes in Computer Science, vol. 2567, Springer, pp. 31–46, 2003.